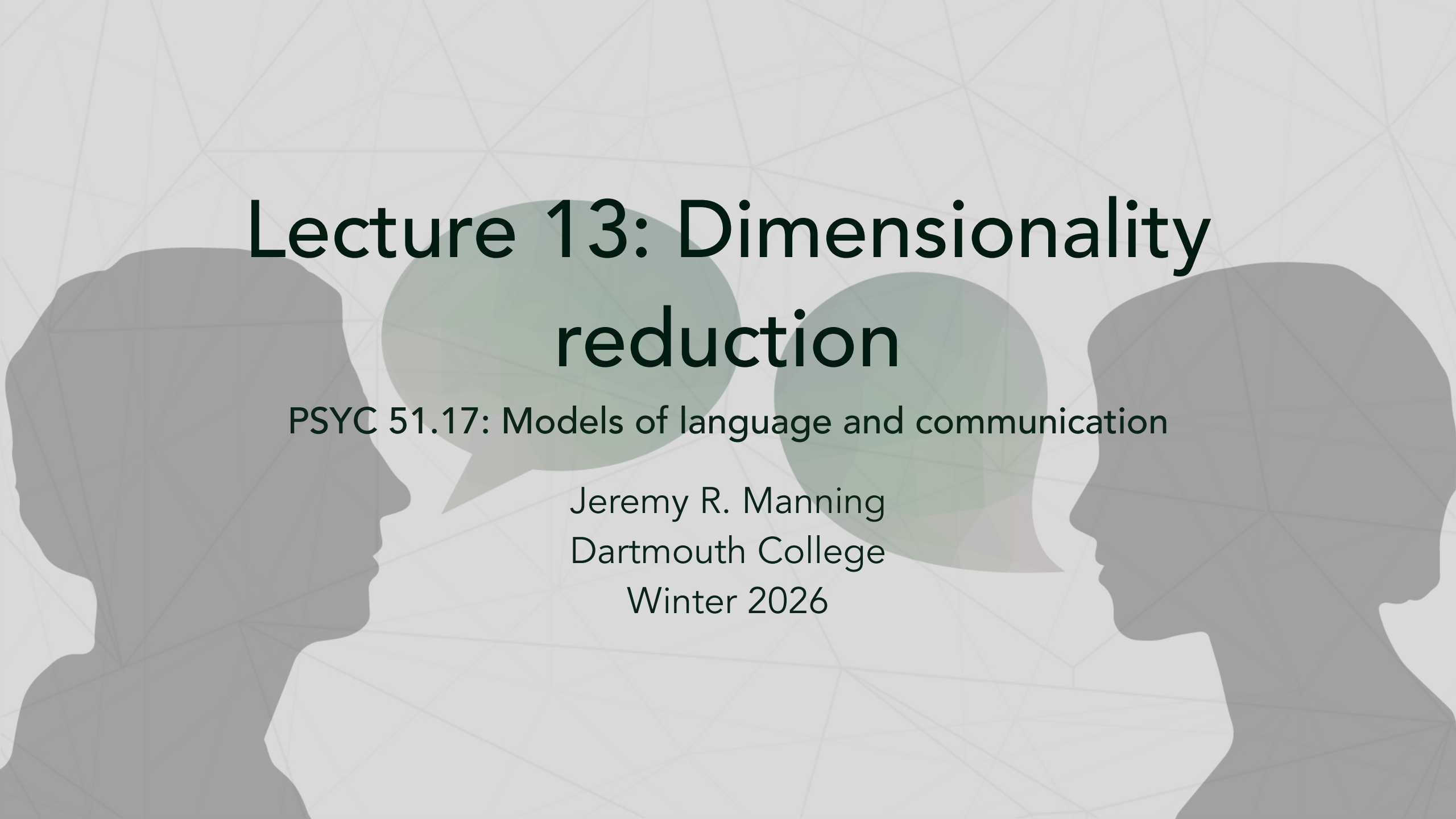




Lecture 13: Dimensionality reduction

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand **why** we need dimensionality reduction for embeddings
2. Know the **matrix factorization** family (PCA, ICA, NMF, etc.)
3. Know the **manifold learning** family (t-SNE, UMAP, MDS, etc.)
4. Know **when to use** each technique
5. Apply **HyperTools** for quick visualization workflows

The goal

Reduce from d dimensions (e.g., 300+) to 2-3 dimensions while preserving meaningful structure for visualization.

The challenge of high dimensions

"To deal with a 14-dimensional space, visualize a 3-D space and say 'fourteen' to yourself very loudly. Everyone does it."

— Geoffrey Hinton

Counterpoint: the curse of dimensionality

High-dimensional spaces behave counter-intuitively: all points tend to be equidistant!

- **Word2Vec / GloVe:** 300 dimensions
- **BERT:** 768 dimensions
- **GPT-3:** 12,288 dimensions

Why dimensionality reduction?

Applications

- **Visualization:** 2D/3D plots to explore semantic structure geometrically
- **Quality assurance:** verify embeddings capture expected relationships
- **Preprocessing:** reduce noise before clustering
- **Interpretation:** understand relationships between words/documents

Two major families

1. **Matrix factorization:** linear methods that decompose data into factors
2. **Manifold learning:** non-linear methods that preserve geometric structure

What is matrix factorization?

Core idea

Decompose the data matrix into a product of smaller matrices:

$$\mathbf{Y} \approx \mathbf{W}\mathbf{F}$$

Where:

- $\mathbf{Y}$ is $n \times d$ (observations $\times$ features)
- $\mathbf{W}$ is $n \times k$ (observations $\times$ components) — the **weights**
- $\mathbf{F}$ is $k \times d$ (components $\times$ features) — the **factors**

Matrix factorization comes in many flavors!

- **PCA**: maximize variance, orthogonal factors
- **ICA**: maximize statistical independence
- **Factor Analysis**: model latent variables with noise
- **NMF**: non-negative constraints (interpretable parts)
- **Dictionary Learning**: sparse, overcomplete representations
- **Topographic Factor Analysis**: spatially organized factors

PCA: Principal Component Analysis

Further reading

Pearson (1901, *Philosophical Magazine*). On lines and planes of closest fit to systems of points in space.

The classic linear method

Finds directions of **maximum variance** and projects data onto these orthogonal "principal components."

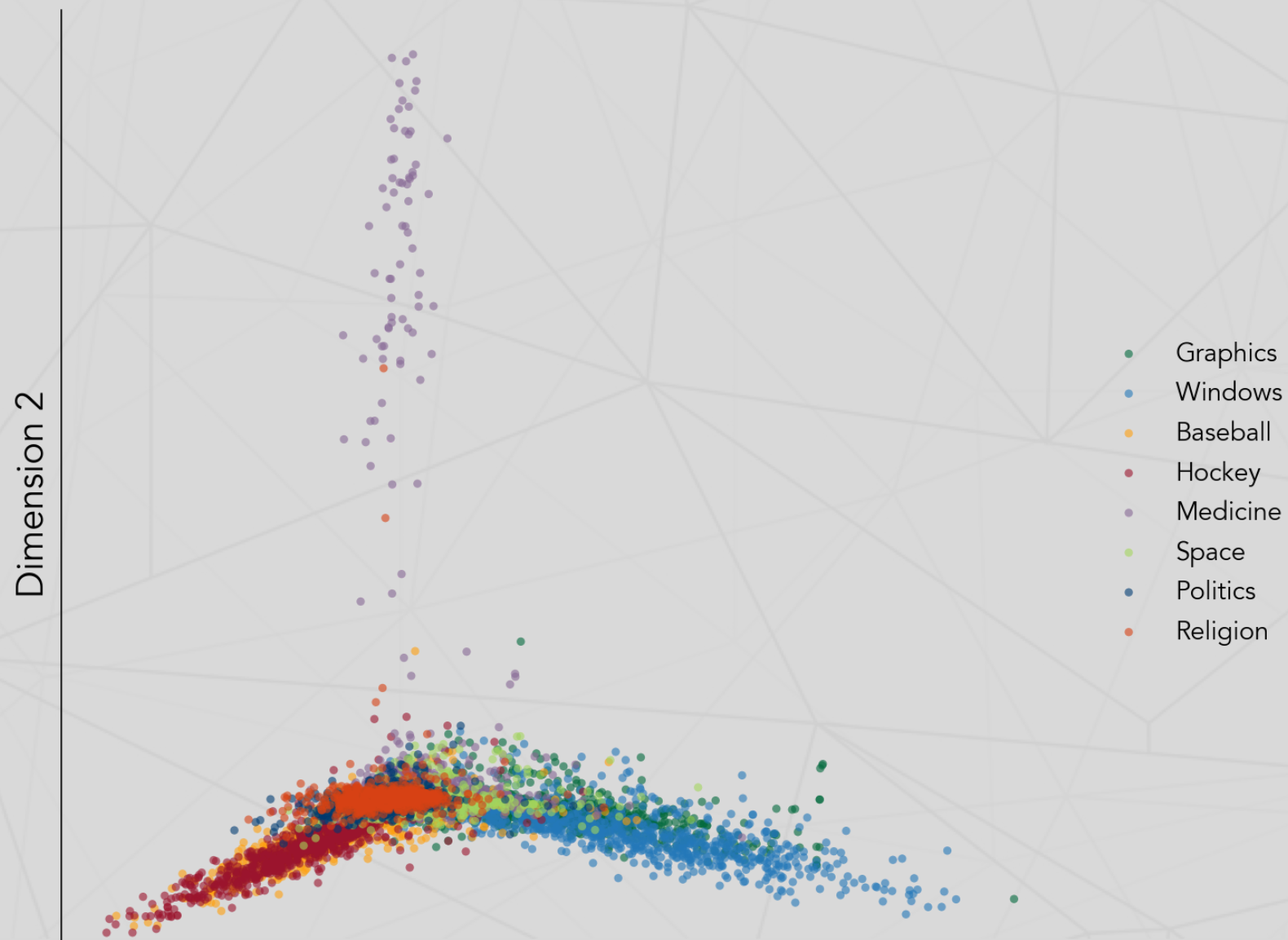
Properties

- **Linear transformation:** preserves global structure
- **Deterministic:** same result every time (for some implementations)
- **Fast:** efficient computation via SVD
- **Interpretable:** PCs ranked by variance explained

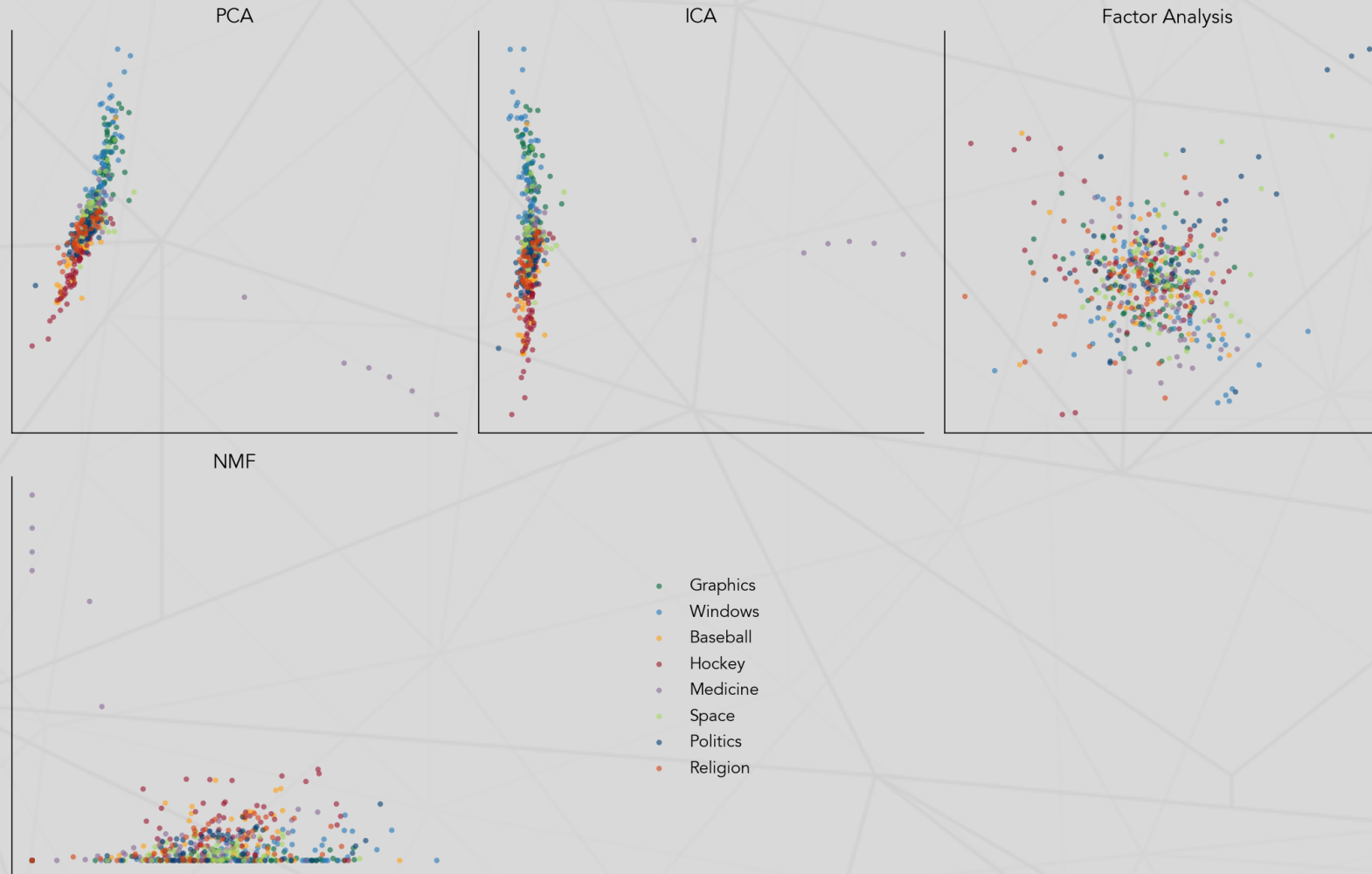
When to use PCA

Quick first exploration, preprocessing for other methods, or when computational speed is critical.

PCA: 20 Newsgroups visualization



Matrix factorization methods compared



t-SNE: t-Distributed Stochastic Neighbor Embedding

Further reading

van der Maaten & Hinton (2008, *JMLR*) Visualizing Data using t-SNE.

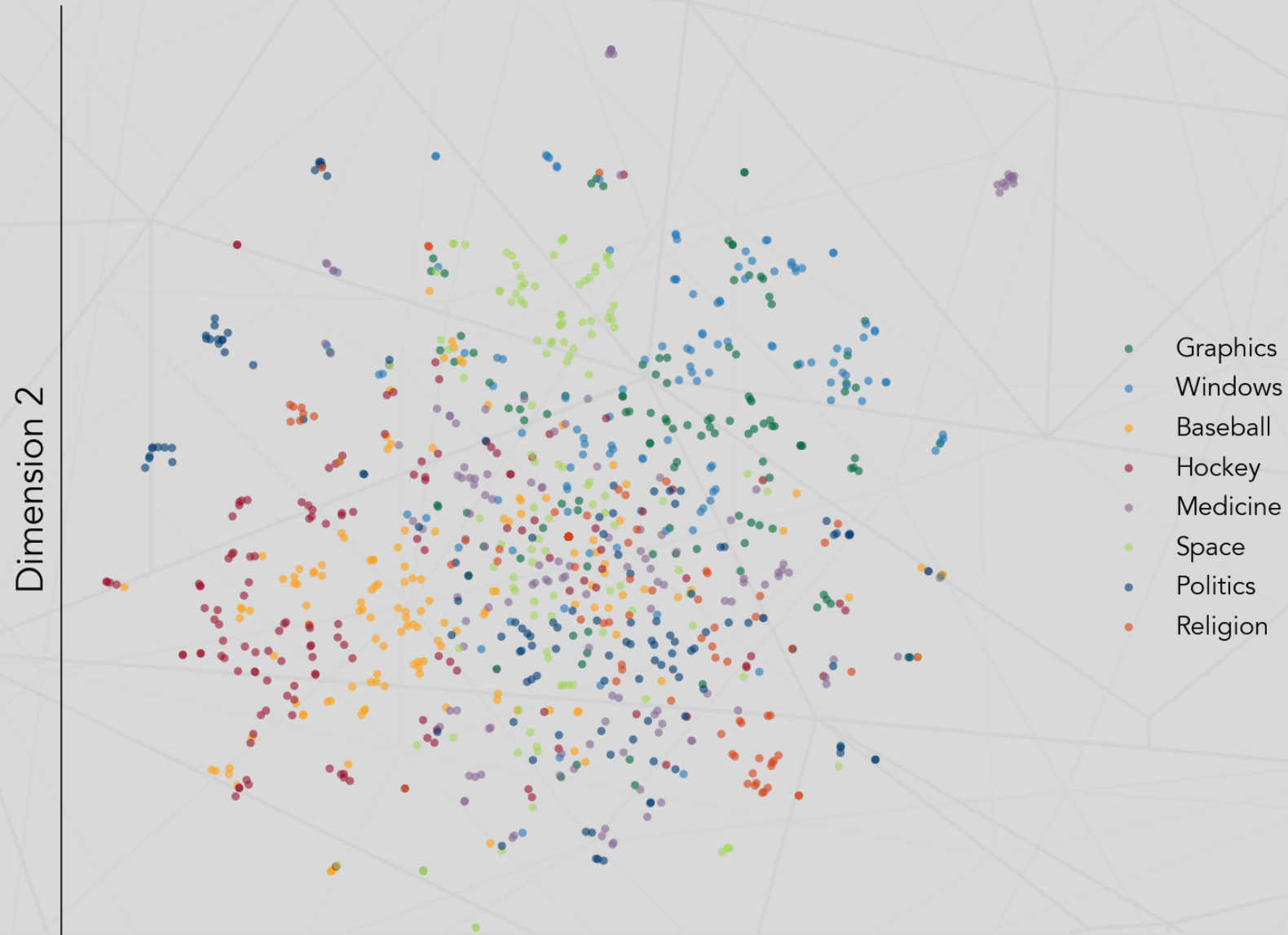
Non-linear visualization

Models similarity as probability distributions. Preserves **local structure** — neighbors stay neighbors.

Caveats

- **Slow:** $O(N^2)$ complexity
- **Non-deterministic:** Different seeds → different layouts
- **Cluster sizes are arbitrary** — don't interpret "bigger" as "more variance"
- **Distances between clusters are meaningless**

t-SNE: 20 Newsgroups visualization



UMAP: Uniform Manifold Approximation & Projection

Further reading

McInnes et al. (2018, [arXiv](#)) UMAP: Uniform Manifold Approximation and Projection.

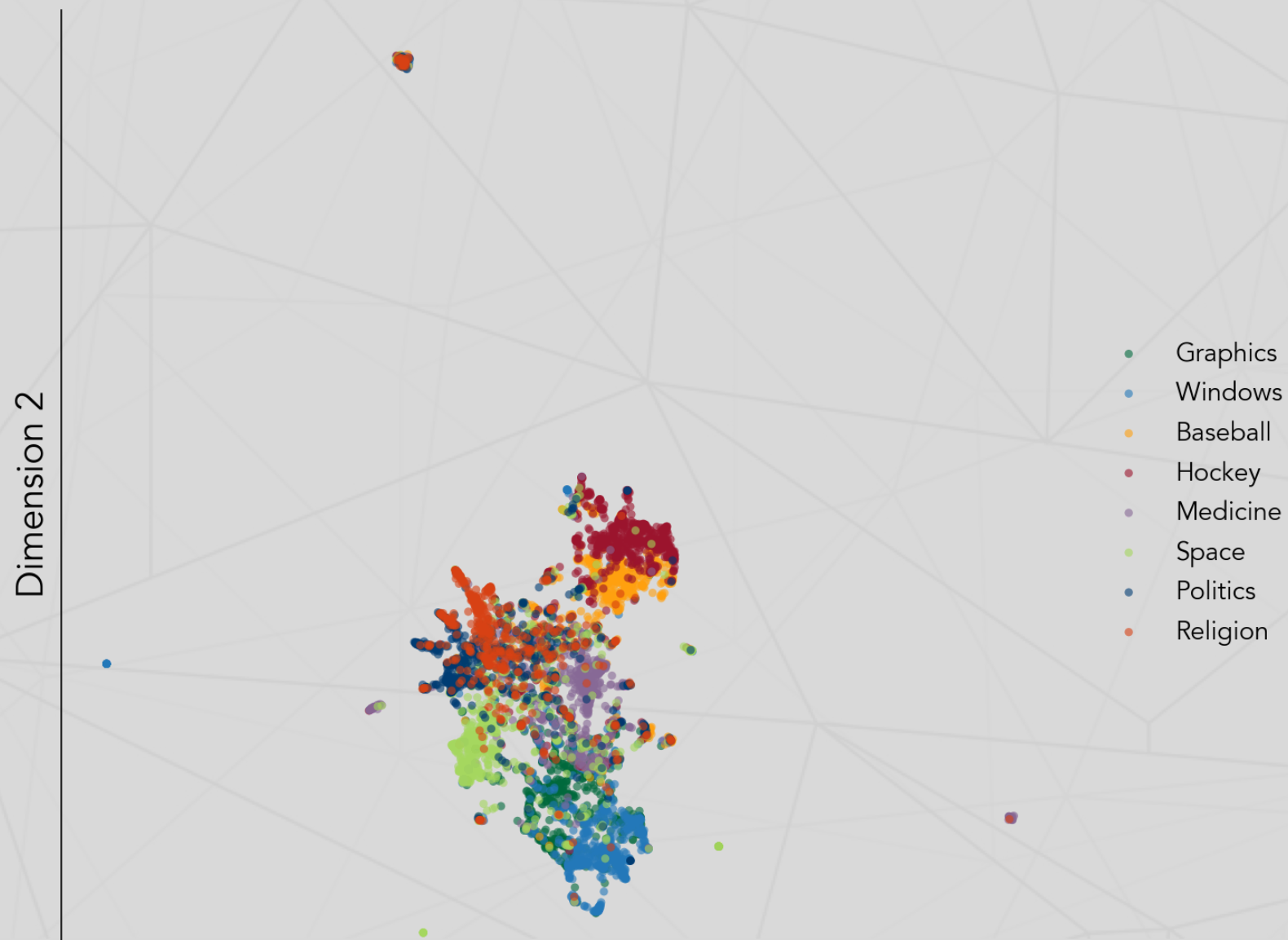
The modern standard (2018)

Theoretically grounded in topology. Preserves **both local AND global** structure.

Key advantages

- **Fast:** $O(N \log N)$ complexity
- **Scalable:** Handles millions of points
- **Out-of-sample:** Can transform new points without refitting
- **Global structure:** Keeps related clusters near each other

UMAP: 20 Newsgroups visualization



Manifold learning: The big picture

Core idea

Assume high-dimensional data lies on a lower-dimensional **manifold** (curved surface). Learn the manifold structure and unfold it.

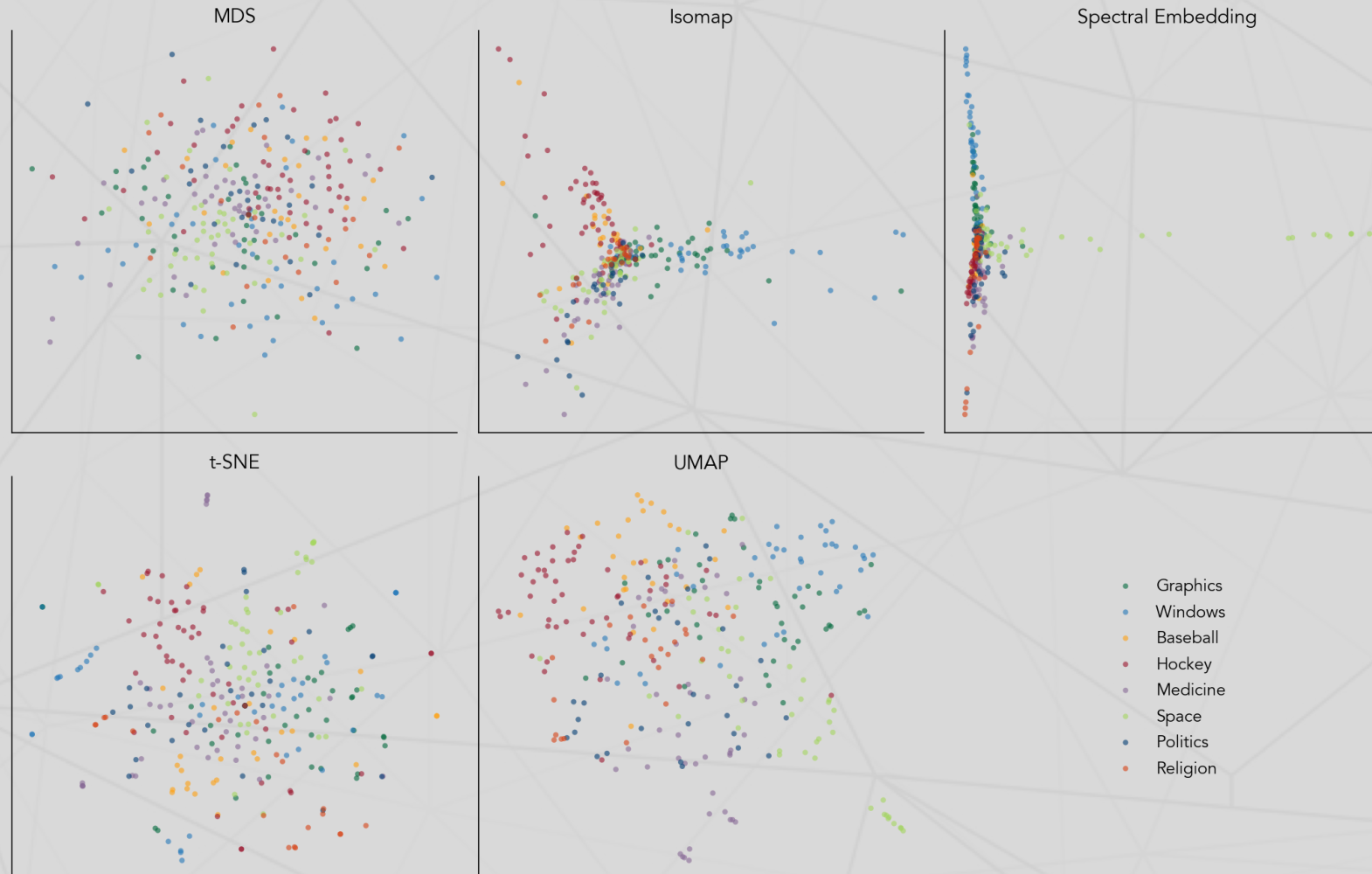
The family

- **MDS**: Preserve pairwise distances
- **Isomap**: Geodesic distances on manifold
- **Spectral Embedding**: Graph Laplacian eigenvectors
- **t-SNE**: Probability-based local structure
- **UMAP**: Topological structure preservation

Key insight

These methods capture **non-linear** relationships that matrix factorization misses.

Manifold learning methods compared



PCA vs. t-SNE vs. UMAP

Feature	PCA	t-SNE	UMAP
Speed	Very Fast	Slow	Fast
Scalability	Excellent	Poor (<10k)	Excellent
Global structure	Yes	No	Yes
Local structure	Partial	Yes	Yes
Embedding			

Visualization best practices

Preparation

- **Standardize:** Use `StandardScaler` for PCA
- **Sample:** For very large datasets, subsample or use PCA first
- **Iterate:** Try multiple hyperparameters

Presentation

- **Color intelligently:** By semantic category, cluster, or feature
- **Interactive:** Use Plotly, datamapplot, or Bokeh for hover tooltips
- **Annotate selectively:** Label representative points, not every one

What you CANNOT conclude

- Exact distances or true high-D geometry
- Quantitative relationships between distant clusters
- Absolute cluster sizes (especially in t-SNE)

HyperTools: Quick visualization workflows

What is HyperTools?

A Python toolbox for **dimensionality reduction-based visual exploration** of high-dimensional data. Reduce, align, and plot in a single function call.

[GitHub](#) | [Documentation](#)



HyperTools: Example code

```
1 import hypertools as hyp
2
3 # Load sample high-dimensional data
4 data = hyp.load('weights').get_data()
5
6 # All-in-one: reduce, align, and plot
7 hyp.plot(data, reduce='UMAP', align='hyper', ndims=3)
8
9 # Or use functions individually
10 reduced = hyp.reduce(data, reduce='PCA', ndims=10)
```

Key functions

- `hyp.plot()` — Visualize with automatic reduction
- `hyp.reduce()` — Dimensionality reduction (PCA, UMAP, t-SNE, etc.)
- `hyp.align()` — Align multiple datasets (Procrustes, hyperalignment)

Summary

What we learned

1. **Curse of Dimensionality:** High-D space is counter-intuitive; reduction essential for visualization
2. **Matrix Factorization:** $\mathbf{Y} \approx \mathbf{WF}$ — PCA, ICA, NMF, Factor Analysis
3. **Manifold Learning:** MDS, Isomap, t-SNE, UMAP — capture non-linear structure
4. **UMAP:** Fast, scalable, preserves local and global structure. The modern default.
5. **HyperTools:** Quick workflow for reduce → align → plot

Try it yourself!



X-hour Notebook — Interactive demo with BERTopic and datamapplot

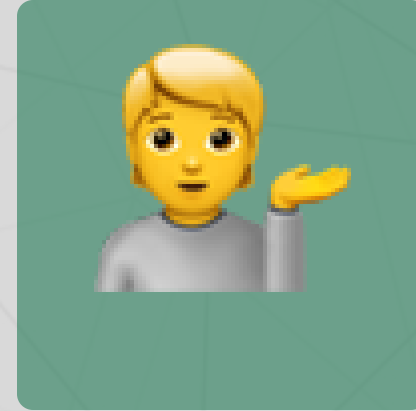
Questions?



Email me



Join our Discord



Come to office hours

Up next

Friday: Cognitive models of semantic representation — how do humans represent meaning?